High Level Design
on
Healthcare Planning Assistant Agent

Course Name: **Agentic AI**

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	ADITYA AKOLKAR	EN22CS301047
2.	AMAN GURJAR	EN22CS301105
3.	Dazzele Dhalani	EN22CS301312
4.	AMAN MAKWANA	EN22CS301106
5.	ADITYA RAJ SISODIYA	EN22CS301063

Group Name: D1

Project Number: AAI-02

Industry Mentor Name:

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2026

Table of Contents

1. Introduction
1.1. Scope of the document
1.2. Intended Audience
1.3. System overview

2. System Design
2.1. Application Design
2.2. Process Flow
2.3. Information Flow
2.4. Components Design
2.5. Key Design Considerations
2.6. API Catalogue

3. Data Design
3.1. Data Model
3.2. Data Access Mechanism
3.3. Data Retention Policies
3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
7.1. Security Aspects
7.2. Performance Aspects

8. References

1. Introduction

The Healthcare planning Assistant is an Agentic AI-based system designed to help users generate a complete healthcare plan based on their medical condition and location. The system uses a Planner Agent that intelligently breaks down the user's request into smaller tasks and coordinates multiple specialized agents, such as a Research Agent, Hospital Finder Agent, and Scheduler Agent.

1.1 Scope of the Document

The purpose of this document is to describe the High-Level Design (HLD) of the *Healthcare Planning Assistant Agent*.

This document provides an overview of the overall system architecture, including major components, their interactions, and the flow of data across the system. It explains how the frontend, backend, agentic AI modules, and external tools work together to generate a complete healthcare plan from user input.

The scope includes:

- System architecture and major components
- Interaction between frontend, backend, and AI agents
- High-level data and process flow
- Integration with external services (LLM APIs, search tools, maps)

1.2 Intended Audience

The intended audience for the Healthcare Planning Assistant includes patients, caregivers, and general users seeking medical guidance. This system helps them by providing a structured healthcare plan based on their disease and location, reducing the effort of searching for reliable treatment options and nearby hospitals. It is especially useful for individuals who want quick, organized, and personalized healthcare information, enabling them to make better decisions about their treatment and manage their health more effectively.

1.3 System overview

The Healthcare Planning Assistant is an Agentic AI system that:

- Accepts user inputs (disease, location)
- Breaks down healthcare tasks using a Planner Agent
- Uses multiple agents:
  - Research Agent → finds treatment info
  - Hospital Finder Agent → finds hospitals
  - Scheduler Agent → creates timeline
- Produces a complete healthcare plan

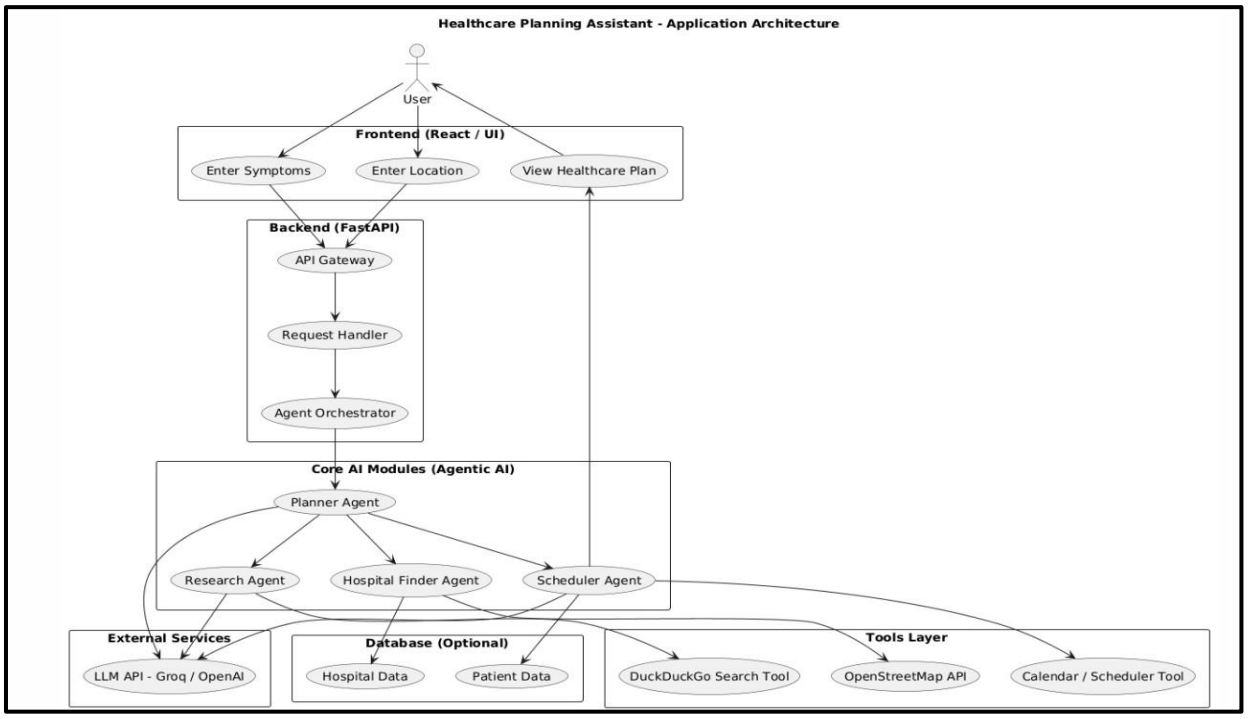
2. System Design

2.1 Application Design

The application follows a multi-agent modular architecture built using:

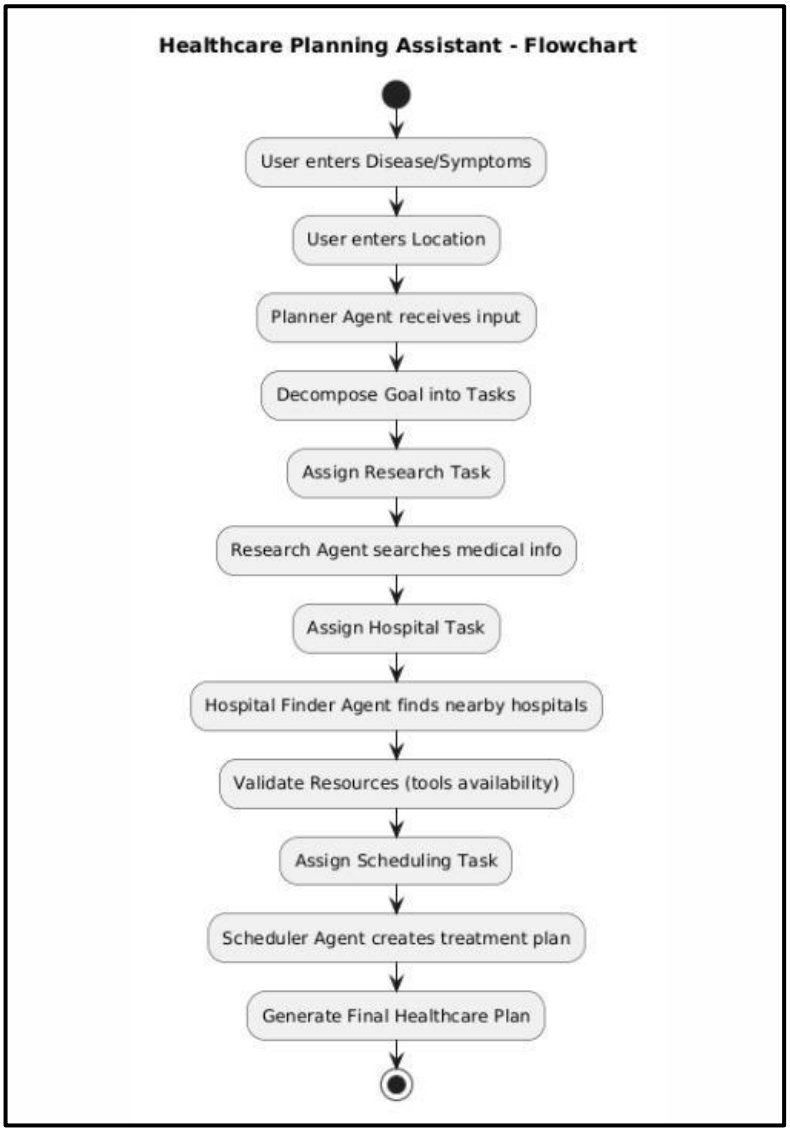
- CrewAI for agent orchestration and coordination
- LangChain for LLM interaction, reasoning, and tool integration
- LLM API (Groq/OpenAI) for intelligent decision-making and planning
- FastAPI as the backend framework for handling requests
- React for the user interface
- Python as the core backend programming language

The system is designed using an Orchestrator–Agent pattern, where a central orchestrator manages the execution flow and coordinates multiple specialized agents.



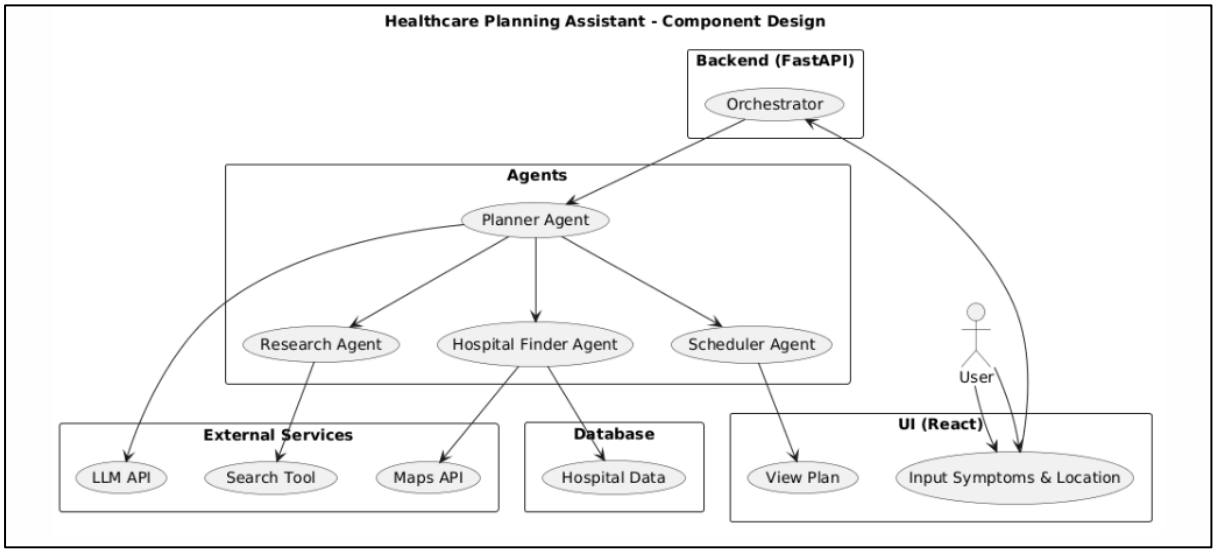
2.2 Process Flow

- **User Input:** User enters symptoms and location
- **Orchestrator Activation:** System starts workflow
- **Planning:** Planner Agent breaks task into steps
- **Research & Hospital Search:** Agents gather medical info and find hospitals
- **Scheduling:** Scheduler Agent creates treatment plan
- **Final Output:** Complete healthcare plan shown to user



2.3 Components Design

- **Orchestrator (Agent Orchestrator):**
  Controls the overall workflow, manages communication between agents, and maintains system state
- **Planner Agent:**
  Analyzes user input and breaks the main goal into smaller tasks
- **Research Agent:**
  Retrieves medical information using LLM and search tools
- **Hospital Finder Agent:**
  Finds nearby hospitals based on user location using maps APIs
- **Scheduler Agent:**
  Creates a structured treatment plan and organizes execution steps
- **LLM API (Groq/OpenAI):**
  Provides intelligence for reasoning, planning, and response generation
- **Tools Layer:**
  Includes search tools, maps API, and scheduler tools for external interaction
- **Database (Optional):**
  Stores hospital data and patient-related information
- **User Interface (React):**
  Allows users to input symptoms/location and view healthcare plan



2.4 Key Design Considerations

- **Agent Specialization:** Each agent handles a specific task
- **Scalability:** Easy to add new agents and features
- **State Management:** Orchestrator controls workflow
- **Data Efficiency:** Fast data retrieval using LLM/tools
- **Communication:** Smooth interaction between agents
- **Maintainability:** Easy to update and manage system

2.6 API Catalogue

Provider	Model	Purpose
CrewAI	Agent Framework	Task orchestration and multi-agent management
LangChain	Framework	LLM integration, reasoning, and tool handling
Groq / OpenAI	LLM API	Reasoning, planning, and response generation
DuckDuckGo API	Search Tool	Fetching medical information from the web
OpenStreetMap API	Maps API	Finding nearby hospitals based on location
FastAPI	Backend Framework	API handling and request processing
React	UI Framework	User interaction and displaying healthcare plan

3. Data Design

3.1 Data Model

The system operates on a retrieval-based structured data model:

```
{
  "symptoms": "string",
  "location": "string",
  "medical_info": "string",
  "hospital_list": ["list"],
  "treatment_plan": "string",
  "final_output": "string"
}
```

3.2 Data Access Mechanism

- Data is retrieved using LLM and search tools
- APIs are used for medical info and hospital data
- Backend processes and formats the data

3.3 Data Retention Policies

- Uses temporary (volatile) storage
- Data is stored only during session and cleared after refresh

3.4 Data Migration

- Not required (stateless system)

4. Interfaces

• **GUI:** React-based web interface
• **External APIs:**

- LLM API (Groq/OpenAI)
- CrewAI (agent orchestration)
- Search & Maps APIs

• **Database:** Optional (hospital/patient data) like SQLite

5. State and Session Management

• Managed by **Orchestrator**
• Maintains workflow state across agents
• Session persists during user interaction

6. Caching

• No dedicated caching used
• Temporary session data acts as cache

7. Non-Functional Requirements

7.1 Security Aspects

• API keys stored using environment variables
• User input handled safely

7.2 Performance Aspects

• Fast processing using agents and tools
• Performance depends on LLM response time
• Efficient task execution via orchestration

8. References

• CrewAI Documentation
• LangChain Documentation
• LLM API (Groq/OpenAI) Doc

Conclusion

The Healthcare Planning Assistant Agent project successfully demonstrates the application of intelligent agent-based systems in solving real-world healthcare planning problems. Using an Agentic AI approach, the system is capable of transforming high-level user inputs such as disease symptoms and location into a structured and actionable healthcare plan.

The designed Planner Agent plays a central role by decomposing complex goals into smaller tasks and coordinating multiple specialized agents, including the Research Agent, Hospital Finder Agent, and Scheduler Agent. These agents collaboratively gather medical information, identify nearby healthcare facilities, and generate a feasible treatment schedule.

The integration of tools such as web search and location-based services ensures that the system provides relevant and practical recommendations. The modular architecture improves scalability, maintainability, and flexibility, allowing future enhancements like real-time appointment booking, integration with hospital APIs, and personalized healthcare recommendations.

Overall, this project highlights how modern AI frameworks like CrewAI and LangChain can be leveraged to build intelligent, autonomous systems that simplify complex decision-making processes. The system not only enhances user convenience but also demonstrates the potential of AI-driven solutions in improving healthcare accessibility and planning.